Module	Main responsibility	Representative services
Feedback and Automation	Rocchio update, recomputation, account policy, auto-apply, reminders, and scheduler	FeedbackService, RocchioService, AutoApplyService, AutomationScheduler
Notification	Policy guard, delivery log, SMTP/no-op mail, and signed email actions	NotificationPolicyGuard, NotificationEmailService, EmailActionService
Content Report	Job/CV reports, access checks, queue grouping, ban/dismiss, counters, and audit	ContentReportService, ContentReportController, AdminReportController
Analytics, Admin, and Audit	Live analytics, account/Job administration, monitoring, and audit persistence	Analytics services, administrative services, audit/common components

The modules cooperate through service calls rather than direct controller-to-repository shortcuts for core workflows. Transaction boundaries belong at service operations that change a consistent group of records. Read-only queries may use dedicated query services or projections to avoid loading unrelated state. DTOs prevent persistence entities from becoming an accidental public API contract.

3.3 Data Design

3.3.1 Core Identity and Recruitment Data

user_account stores identity, role, activation, verification, language, and password hash. A Candidate owns a profile, multiple CVs, reviewable CV sections, extracted skills, portfolio links, and portfolio projects. A Recruiter owns an employer profile and Jobs. A Job stores its JD, structured recruitment fields, lifecycle status, report count, urgent flag, deadline, application count, source details, and vector data. A CV also stores its lifecycle status and pending report count.

matching is the unique association between one CV and one Job. It stores the direct score, label, separate Potential flag and explanation, reasons, and recomputation state. application links a Candidate and Job and may reference the CV and Matching used at that time. feedback stores a judgment about a Matching. recruiter_cv_bookmark keeps a Recruiter's saved Candidate/CV for an owned Job, while recommendation_interaction records events such as viewed, skipped, applied, saved, not interested, or show similar.

3.3.2 Automation, Communication, and Operational Data

automation_policy stores one policy per account, including enable/pause state, thresholds, quotas, cooldowns, quiet hours, category guards, and notification preferences. email_action stores expiring one-time actions using a token hash. content_report stores the reporter, JOB or CV target, reason, comment, pending/resolved state, resolving administrator, note, and timestamps. Notification delivery records support policy decisions, while audit_log records important user, moderation, and automated actions.

Table 3.2. Key integrity constraints

Constraint	**Purpose**
Unique normalized user email	Prevent duplicate identities that differ only by case
One role profile and one policy per account	Preserve ownership and account-level settings
Partial unique default CV per Candidate	Ensure at most one active default CV
Unique Matching per CV and Job	Prevent duplicate direct scores for the same pair
Unique Application per Candidate and Job	Prevent duplicate applications or invitations
Unique bookmark and feedback keys	Keep one bookmark/judgment for each defined actor-target relation
Unique pending report per reporter and target	Prevent duplicate unresolved abuse reports from the same account
Check constraints on roles, states, labels, report values, salary, and channels	Reject invalid domain values at the database boundary
Indexes for Jobs, ownership, ranking, audit, and report queue	Support catalogues, reminders, matching, and moderation queries

Flyway migrations are the authoritative schema history. Hibernate uses ddl-auto=validate, so the application checks entity-to-schema compatibility instead of silently generating a different database. Indexes support active Jobs, ownership, matching, applications, feedback, tokens, audit chronology, and the pending report queue. A partial unique index prevents one reporter from creating duplicate pending reports for the same target.

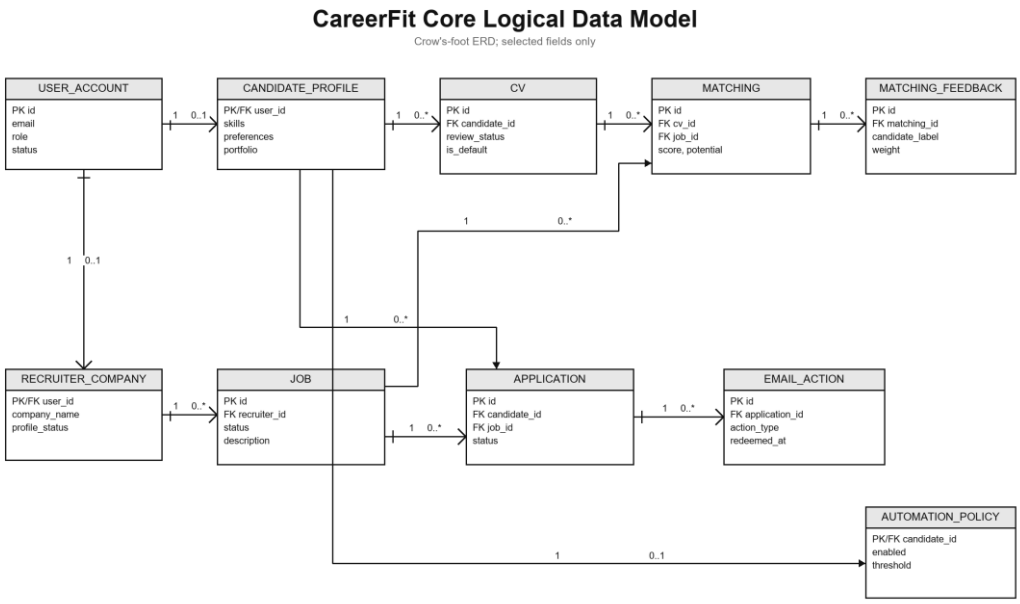

Figure 3.2. CareerFit core logical data model

3.4 Security, Failure, and Consistency Design

3.4.1 Authentication and Authorization

The backend is stateless at the HTTP session layer. A JWT authentication filter runs before username/password authentication processing, and a user-ID resolution filter runs after JWT validation. Public access is explicitly limited to authentication entry points, selected job/employer/analytics GET endpoints, health/metrics/API documentation, and email-action redemption. Candidate, Recruiter, and Administrator endpoint groups require corresponding roles. Automation requires authentication, with service logic responsible for policy ownership.

The security configuration disables CSRF because the application uses stateless bearer authentication, configures a CORS origin allow-list, denies framing, applies a content-security policy, and configures HSTS. These headers do not replace TLS termination in the deployed environment. Public Swagger and Prometheus access are acceptable for local development but should be restricted in a production deployment.

3.4.2 Failure Handling

CV processing represents long-running work with statuses rather than keeping a browser request open until every score completes. Extraction or validation failure records a failure state and reason. Matching tasks start after commit so background work cannot read uncommitted rows. Report moderation uses transactions and row locking to update the target, report counter, report resolutions, and audit state together. A banned Job is excluded from active flows, while a banned CV can no longer remain the default CV.

Email can operate through SMTP or a no-op implementation. Notification delivery outcomes are recorded as sent, skipped, or failed, and policy guards enforce deduplication, quota, cooldown, and timing rules. Email failure must not roll back an already valid matching result. Conversely, application creation and status changes use transactions and uniqueness constraints because partial persistence would change recruitment state.

3.4.3 Identified Design Risks

Table 3.3. Design risks and required treatment

Risk	**Current control**	**Remaining treatment**
Unauthorized cross-account access	URL roles and service ownership/visibility checks	Keep negative integration tests for identifier-based operations
Duplicate applications, Matchings, or reports	Service checks plus unique constraints/indexes	Convert database conflicts into stable API errors
False or abusive content reports	Fixed reasons, one pending report per reporter-target, and Administrator review	Add rate limits, evidence rules, moderator assignment, and abuse monitoring
Incorrect moderation action	Case detail, explicit Ban/Dismiss, transactions, and audit events	Add appeal, restoration, and dual-review policy for production
Replayed email action	Hashed token, pending/redeemed state, expiry, and POST execution	Retain replay, expiry, scanner, and rate-limit tests
Scheduler/configuration drift	Scheduler timing reads application properties	Maintain configuration and schedule-boundary tests
Sensitive data in logs	Structured audit metadata and DTO boundaries	Review retention, redaction, access, and exported evidence